Лабораторная работа № 2

Проектирование лексического анализатора

Вариант 6 · логические выражения с константами 0 и 1

Цель работы

Получить практические навыки построения сканера: выделять лексемы входного текста, распознавать их типы, формировать таблицу лексем и выдавать сообщения о лексических ошибках.

1. Задание по лабораторной работе

Входной язык содержит логические выражения, разделённые символом ; (точка с запятой). Логические выражения состоят из идентификаторов, констант 0 и 1, знака присваивания :=, знаков операций or, xor, and, not и круглых скобок.

Ограничение: длина идентификатора — не более 32 символов. Комментарии допускаются неограниченной длины; в реализации выбрана форма {…}. Пробельные символы и комментарии удаляются сканером. При обнаружении ошибки сканирование продолжается, чтобы вывести все найденные ошибки.

Примечание о приложенных материалах: PDF «2_5 …» содержит учебное условие и варианты, PDF «2_1 Теория» — методические сведения и пример автомата. Lab2.txt и Lab2.exe являются примером реализации, а не отдельным условием варианта 6.

2. КС-грамматика входного языка

Синтаксические правила записаны в расширенной форме Бэкуса—Наура. В грамматике задан приоритет операций: not выше and, затем xor, затем or.

```
<Программа> ::= <Выражение> { ; <Выражение> } [ ; ]
<Выражение> ::= <Идентификатор> := <OrExpr> | <OrExpr>
<OrExpr> ::= <XorExpr> { or <XorExpr> }
<XorExpr> ::= <AndExpr> { xor <AndExpr> }
<AndExpr> ::= <UnaryExpr> { and <UnaryExpr> }
<UnaryExpr> ::= not <UnaryExpr> | <Первичный>
<Первичный> ::= <Идентификатор> | 0 | 1 | ( <OrExpr> )

<Идентификатор> ::= <Буква> { <Буква> | <Цифра> | _ }      (длина ≤ 32)
<Буква> ::= A | ... | Z | a | ... | z
<Цифра> ::= 0 | 1 | ... | 9
```

Лексические классы

Лексема	Тип	Условие распознавания
or, xor, and, not	логическая операция	точное совпадение строчной записи
0, 1	логическая константа	один символ; последовательности 00, 01, 10 и т. п. ошибочны
:=	знак присваивания	два соседних символа
;	разделитель	один символ
(,)	скобка	один символ
буква…	идентификатор	латинская буква, затем буквы/цифры/_, максимум 32
{…}	комментарий	неограниченная длина, включая переводы строк; не попадает в таблицу

3. Алгоритм и конечный автомат

Сканер выполняет один проход слева направо. Сначала пропускаются пробелы и комментарии. Затем по первому символу выбирается распознаватель: слово, числовая последовательность, присваивание, разделитель, скобка или ошибка. После выделения лексема проверяется, заносится в таблицу, а позиция строки и столбца обновляется. Ошибочный фрагмент поглощается до естественной границы, после чего анализ продолжается.

На графе ниже qID и qNUM — допускающие состояния. В qID после распознавания выполняется классификация: точные строки or/xor/and/not становятся операциями, остальные — идентификаторами. Состояние ER означает ошибку. qCOM допускает любой символ до закрывающей скобки комментария.

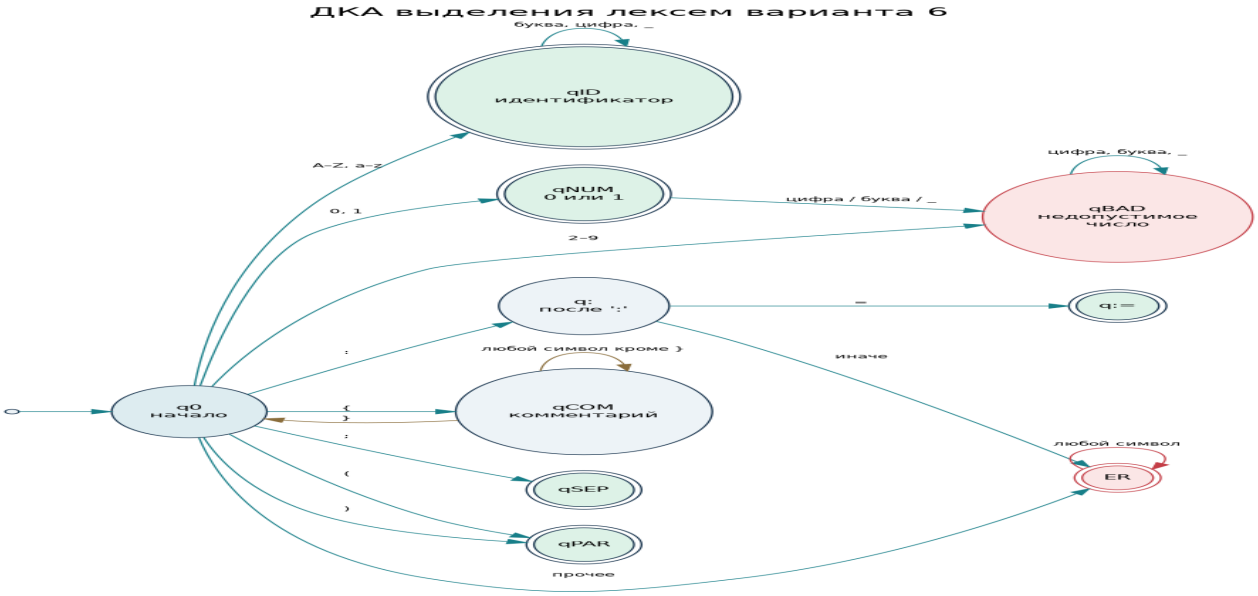


Рисунок 1 — ДКА выделения лексем варианта 6

4. Результаты работы программы

Программа запускается командой: python lab2_lexer.py <входной_файл> [--csv table.csv] [--json result.json]. Код возврата равен 0 при отсутствии лексических ошибок, 1 при наличии ошибок и 2 при ошибке чтения файла.

4.1. Корректный пример

```
{ Пример корректного входного текста }
a := not (b and 1) or c xor 0;
flag := a and not b;
result := (x or y) and (not z xor 1)
```

Распознано лексем: 34; лексических ошибок: 0.

№	Лексема	Тип	Строка	Столбец
1	a	Идентификатор	2	1
2	:=	Знак присваивания	2	3
3	not	Логическая операция	2	6
4	(	Левая скобка	2	10
5	b	Идентификатор	2	11
6	and	Логическая операция	2	13
7	1	Логическая константа	2	17
8	)	Правая скобка	2	18
9	or	Логическая операция	2	20
10	c	Идентификатор	2	23
11	xor	Логическая операция	2	25
12	0	Логическая константа	2	29
13	;	Разделитель выражений	2	30
14	flag	Идентификатор	3	1
15	:=	Знак присваивания	3	6
16	a	Идентификатор	3	9
17	and	Логическая операция	3	11
18	not	Логическая операция	3	15
19	b	Идентификатор	3	19
20	;	Разделитель выражений	3	20

Показаны первые 20 строк; полная таблица из 34 лексем сохранена в valid_variant6_tokens.csv.

4.2. Проверка ошибок

```
a := 10 or b; { число 10 недопустимо }
very_long_identifier_12345678901234567890 := 0;
bad@name := 1; _wrong := 0; c : 1; d := 2;
{ незавершённый комментарий
```

В тесте обнаружено ошибок: 7. Проверяются недопустимое число 10, идентификатор длиннее 32 символов, недопустимый символ @, идентификатор с начальным _, ошибочное присваивание ':' и незавершённый комментарий.

Строка	Столбец	Сообщение	Фрагмент
1	6	допустимы только логические константы 0 и 1	'10'
2	1	длина идентификатора превышает 32 символа	'very_long_identifier_12345678901234567890'
3	4	недопустимый символ входного языка	'@'
3	16	идентификатор должен начинаться с латинской буквы	'_wrong'
3	31	ожидался символ '=' после ':'	':'
3	41	допустимы только логические константы 0 и 1	'2'
4	1	незавершённый комментарий: отсутствует '}'	'{ незавершённый комментарий\n'

5. Текст программы

Ниже приведён полный текст файла lab2_lexer.py. Он также приложен отдельно в репозитории вместе с тестами и примерами входных данных.

```
001 #!/usr/bin/env python3
002 """Lexical analyzer for Laboratory work 2, variant 6.
003
004 The scanner recognizes the token alphabet from the assignment.  It deliberately
005 does not perform syntax analysis: malformed token text is reported while the
006 remaining input is scanned so that one run produces a complete token table.
007 """
008 from __future__ import annotations
009
010 import argparse
011 import csv
012 import json
013 import re
014 import sys
015 from dataclasses import asdict, dataclass
016 from pathlib import Path
017
018 MAX_IDENTIFIER_LENGTH = 32
019 KEYWORDS = {"or", "xor", "and", "not"}
020 IDENTIFIER_RE = re.compile(r"[A-Za-z][A-Za-z0-9_]*")
021 WORD_RE = re.compile(r"[A-Za-z0-9_]+")
022
023
024 @dataclass
025 class Token:
026     number: int
027     lexeme: str
028     kind: str
029     value: str
030     line: int
031     column: int
032
033
034 @dataclass
035 class LexicalError:
036     message: str
037     lexeme: str
038     line: int
039     column: int
040
041
042 class Lexer:
043     """Single-pass scanner with line/column tracking and error recovery."""
044
045     def __init__(self, source: str):
046         self.source = source
047         self.i = 0
048         self.line = 1
049         self.column = 1
050         self.tokens: list[Token] = []
051         self.errors: list[LexicalError] = []
052
053     def _advance(self) -> str:
054         ch = self.source[self.i]
055         self.i += 1
056         if ch == "\n":
057             self.line += 1
058             self.column = 1
059         else:
060             self.column += 1
061         return ch
062
063     def _take(self, start: int) -> str:
064         return self.source[start:self.i]
065
066     def _add_token(self, lexeme: str, kind: str, value: str, line: int, column: int) -> None:
067         self.tokens.append(Token(len(self.tokens) + 1, lexeme, kind, value, line, column))
068
069     def _error(self, message: str, lexeme: str, line: int, column: int) -> None:
070         self.errors.append(LexicalError(message, lexeme, line, column))
071
072     def scan(self) -> tuple[list[Token], list[LexicalError]]:
073         while self.i < len(self.source):
074             ch = self.source[self.i]
075             if ch.isspace():
076                 self._advance()
077                 continue
078             start, line, column = self.i, self.line, self.column
079             if ch == "{":
080                 self._scan_comment(start, line, column)
081             elif ch.isalpha() and ch.isascii():
082                 self._scan_word(start, line, column)
083             elif ch.isdigit():
084                 self._scan_digit_run(start, line, column)
085             elif ch == "_":
086                 self._scan_bad_word(start, line, column, "идентификатор должен начинаться с латинской буквы")
087             elif ch == ":":
088                 self._advance()
089                 if self.i < len(self.source) and self.source[self.i] == "=":
090                     self._advance()
091                     self._add_token(":=", "Знак присваивания", ":=", line, column)
092                 else:
093                     self._error("ожидался символ '=' после ':'", ":", line, column)
094             elif ch in ";()":
095                 self._advance()
096                 names = {";": "Разделитель выражений", "(": "Левая скобка", ")": "Правая скобка"}
```

```
097             self._add_token(ch, names[ch], ch, line, column)
098         elif ch == "}":
099             self._advance()
100             self._error("закрывающая скобка комментария без открывающей", ch, line, column)
101         else:
102             self._advance()
103             self._error("недопустимый символ входного языка", ch, line, column)
104     return self.tokens, self.errors
105
106 def _scan_comment(self, start: int, line: int, column: int) -> None:
107     self._advance()  # {
108     while self.i < len(self.source) and self.source[self.i] != "}":
109         self._advance()
110     if self.i < len(self.source):
111         self._advance()  # }
112     else:
113         self._error("незавершённый комментарий: отсутствует '}'", self.source[start:self.i], line, column)
114
115 def _scan_word(self, start: int, line: int, column: int) -> None:
116     match = WORD_RE.match(self.source, self.i)
117     assert match is not None
118     while self.i < match.end():
119         self._advance()
120     lexeme = self._take(start)
121     if lexeme in KEYWORDS:
122         self._add_token(lexeme, "Логическая операция", lexeme, line, column)
123     elif len(lexeme) > MAX_IDENTIFIER_LENGTH:
124         self._error(f"длина идентификатора превышает {MAX_IDENTIFIER_LENGTH} символа", lexeme, line, column)
125     else:
126         self._add_token(lexeme, "Идентификатор", lexeme, line, column)
127
128 def _scan_digit_run(self, start: int, line: int, column: int) -> None:
129     match = WORD_RE.match(self.source, self.i)
130     assert match is not None
131     while self.i < match.end():
132         self._advance()
133     lexeme = self._take(start)
134     if lexeme in {"0", "1"}:
135         self._add_token(lexeme, "Логическая константа", lexeme, line, column)
136     else:
137         self._error("допустимы только логические константы 0 и 1", lexeme, line, column)
138
139 def _scan_bad_word(self, start: int, line: int, column: int, message: str) -> None:
140     match = WORD_RE.match(self.source, self.i)
141     assert match is not None
142     while self.i < match.end():
143         self._advance()
144     self._error(message, self._take(start), line, column)
145
146
147 def scan_file(path: Path) -> tuple[list[Token], list[LexicalError]]:
148     return Lexer(path.read_text(encoding="utf-8")).scan()
149
150
151 def render_table(tokens: list[Token]) -> str:
152     headers = ("№", "Лексема", "Тип", "Значение", "Строка", "Столбец")
153     rows = [headers] + [tuple(str(getattr(t, field)) for field in ("number", "lexeme", "kind", "value", "line", "column")) for t in tokens]
154     widths = [max(len(row[i]) for row in rows) for i in range(len(headers))]
155     line = "  ".join("-" * width for width in widths)
156     out = ["  ".join(value.ljust(widths[i]) for i, value in enumerate(headers)), line]
157     out.extend("  ".join(value.ljust(widths[i]) for i, value in enumerate(row)) for row in rows[1:])
158     return "\n".join(out)
159
160
161 def write_csv(path: Path, tokens: list[Token]) -> None:
162     with path.open("w", encoding="utf-8-sig", newline="") as handle:
163         writer = csv.DictWriter(handle, fieldnames=["number", "lexeme", "kind", "value", "line", "column"])
164         writer.writeheader()
165         writer.writerows(asdict(token) for token in tokens)
166
167
168 def main(argv: list[str] | None = None) -> int:
169     parser = argparse.ArgumentParser(description="Лексический анализатор, лабораторная работа №2, вариант 6")
170     parser.add_argument("input", type=Path, help="входной текстовый файл")
171     parser.add_argument("--csv", type=Path, help="сохранить таблицу лексем в CSV")
172     parser.add_argument("--json", type=Path, help="сохранить результат анализа в JSON")
173     args = parser.parse_args(argv)
174     try:
175         tokens, errors = scan_file(args.input)
176     except (OSError, UnicodeError) as exc:
177         print(f"Ошибка чтения файла: {exc}", file=sys.stderr)
178         return 2
179     print(f"Файл: {args.input}")
180     print(f"Распознано лексем: {len(tokens)}")
181     print(render_table(tokens))
182     if errors:
183         print(f"\nЛексические ошибки: {len(errors)}", file=sys.stderr)
184         for error in errors:
185             print(f"  строка {error.line}, столбец {error.column}: {error.message}; фрагмент {error.lexeme!r}", file=sys.stderr)
186     else:
187         print("\nЛексических ошибок не обнаружено.")
188     if args.csv:
189         write_csv(args.csv, tokens)
190     if args.json:
191         args.json.write_text(json.dumps({"tokens": [asdict(t) for t in tokens], "errors": [asdict(e) for e in errors]}, ensure_ascii=False, indent=2), encoding="utf-8")
192     return 1 if errors else 0
193
194
195 if __name__ == "__main__":
196     raise SystemExit(main())
```

6. Выводы

В ходе работы разработан лексический анализатор для варианта 6. Он распознаёт идентификаторы, логические константы 0 и 1, операции or/xor/and/not, присваивание :=, разделители и круглые скобки, пропускает комментарии {…}, формирует таблицу лексем с позициями и сообщает об ошибках. Обработка продолжается после ошибки, поэтому пользователь получает полный список проблем во входном файле.

Корректность проверена модульными тестами: распознавание всех классов лексем, переход строк после многострочного комментария, восстановление после ошибок и ограничение длины идентификатора. Дополнительно обработан приложенный пример Lab2.txt: сканер сформировал таблицу и обнаружил ошибки, соответствующие ограничениям варианта 6.

Исходный код, отчёт, схема автомата, тесты и примеры входных файлов находятся в архиве проекта.